App Development Project Report

1. Student Details

Name: Kandepu Charishma
Roll Number: 24f2002804
Email: 24f2002804@ds.study.iitm.ac.in
About Me: I am a student at IIT Madras BS Degree program with an interest in business, data, product management and networking. I am a curious individual and like to keep learning new things.

---

2. Project Details

Project Title: TrekTrail – Trekking Management Application

Problem Statement:

To design and build a Trekking Management Application that helps adventure organizations manage trek activities, bookings, and staff coordination through a role-based web system.

Approach:

The app was built using Flask as the backend framework with a modular, blueprint-based structure separating Admin, Trek Staff, and Trekker functionality. It allows Admin to manage treks and staff, Trek Staff to update and monitor their assigned treks, and Trekkers to browse, book, and track treks, with role-based access control, real-time slot management to prevent overbooking, and Chart.js visualizations to track bookings and participation trends across dashboards.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, and improving variable naming consistency.

The extent of AI/LLM usage is around **10-15%**, limited to **code suggestions and specific code review**.

All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for interacting with the SQLite database
Flask-Login	Session-based authentication and role-based access control
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and layout
Chart.js	Dashboard visualizations
SQLite	Lightweight local database

---

5. Database Schema Design

Tables:

1. User : id, full_name, email, password_hash, role (admin/staff/trekker), is_blacklisted, created_at. Stores all three account types in one table, differentiated by role.
2. StaffProfile : id, user_id (FK → User), contact_number, status (pending/approved/blacklisted), approved_at, created_at
3. Trek : id, name, location, difficulty, duration_days, total_slots, available_slots, assigned_staff_id (FK → StaffProfile), status (Pending/Approved/Open/Closed/Completed), start_date, end_date, description, created_at
4. Booking : id, user_id (FK → User), trek_id (FK → Trek), booking_date, status (Booked/Cancelled/Completed), payment_status, created_at

Relationships:

- One-to-One → User (staff) → StaffProfile
- One-to-Many → StaffProfile → Trek

- One-to-Many → User → Booking
- One-to-Many → Trek → Booking

Design reasoning: A single User table with a role column keeps authentication logic simple and centralized, while StaffProfile is kept separate so staff-specific fields (approval status, contact number) don't clutter the core user model or apply to trekkers/admins. available_slots is tracked independently from total_slots so overbooking checks and cancellations can be handled with simple increment/decrement logic rather than recalculating from booking counts each time.

---

6. Architecture and Features

Architecture Overview:

- run.py — application entry point
- config.py — configuration (database URI, secret key)
- app/__init__.py — application factory, extension setup, blueprint registration
- app/models.py — SQLAlchemy models (User, StaffProfile, Trek, Booking)
- app/decorators.py — custom role_required decorator for access control
- app/routes/ — Flask Blueprints, one per role (auth.py, admin.py, staff.py, trekker.py)
- app/templates/ — Jinja2 HTML templates, organized into subfolders per role
- seed_admin.py — script to programmatically create tables and the pre-existing Admin account

Implemented Features:

- Role-based registration and login (Admin pre-seeded, Staff/Trekker self-register)

- Staff accounts require Admin approval before dashboard access
- Staff dashboard restricted to only their assigned treks, with slot/status updates and participant lists
- Trekker dashboard with trek browsing, search, difficulty/location filters, and booking
- Overbooking and duplicate-booking prevention enforced at the route level
- Search by name or ID across treks, staff, and users (Admin)
- Chart.js visualizations across all three dashboards (Admin, Staff and Trekker)

---

7. Video Presentation

Drive Link:
https://drive.google.com/file/d/1agzCL7iZBAZ-WxmZ8VwpThsYToDEvSRJ/view?usp=sharing

---